

flutter介绍以及实战

Flutter是什么？

Flutter是谷歌的移动端跨平台UI框架，可以快速在iOS和Android上构建高质量的原生用户界面。Flutter可以与现有的代码一起工作。在全世界，Flutter正在被越来越多的开发者和组织使用，并且Flutter是完全免费、开源的。简单来说，Flutter是一款移动应用程序SDK，包含框架、控件和一些工具，可以用一套代码同时构建Android和iOS应用，并且性能可以达到原生应用一样的性能。

Flutter开发前期准备

安装Flutter +配置编辑器<https://flutterchina.club/get-started/install/>

flutter doctor 遇到的坑

坑1: flutter Unable to find bundled Java version

解决方案: <https://blog.csdn.net/wh5359/article/details/118895172>

安装Android studio mac版本教程以及踩坑记录

下载地址: <https://developer.android.com/studio>

下载中选择

根据你的电脑的芯片来选择

- intel 芯片：选择 Mac with Intel chip
- m1 芯片: 选择 Mac with Apple chip

Select the version of Android Studio that's right for your Mac:

Android Studio Bumblebee 2021.1.1 Patch 2

Mac with Intel chip

Mac with Apple chip

安装教程: <https://www.jianshu.com/p/2d1357203f09>

在macOS上搭建Flutter开发环境:<https://flutterchina.club/setup-macos/>

Mac 环境变量设置不生效:

发现设置jdk环境变量时需要每次都source才能用

原因: 使用的zsh

解决方法: <https://www.jianshu.com/p/24bacb0f3401>

新建flutter项目

下载flutter sdk :<https://docs.flutter.dev/development/tools/sdk/releases?tab=macos#macos>

安装flutter sdk :https://blog.csdn.net/weixin_44653603/article/details/88019791

安装sdk 会遇到的问题 :

<https://blog.csdn.net/feiyue0823/article/details/101116651>

一些常用组件

StatelessWidget StatefulWidget

实现Flutter App时, 我们使用Widgets构建UI, 这些widgets有两种类型, stateful和stateless, 由于整个APP全部使用widget构建, 所以在构建每个widget时, 都需要判断使用哪种状态, 这就要求必须对状态有深入了解, 才能确保每个决定都是准确无误的。

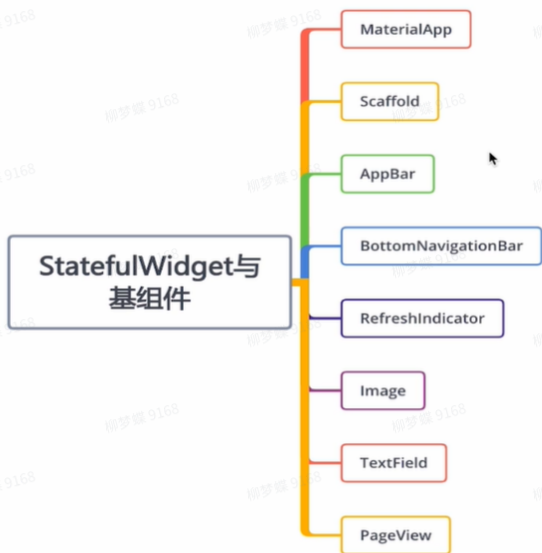
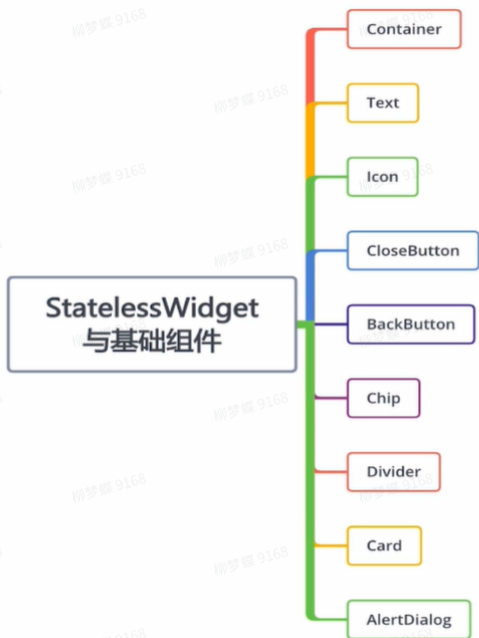
什么时候用stateless部件呢?

- 1, 可能创建一个自定义的进度控件, 这个控件仅使用初始化的属性来展示给用户, 这样就不需要持有任何状态, 因为父部件会控制其是否显示, 这样父部件就管理了这个widget本身是否显示的状态了。
- 2, 你可能创建一个用在列表中的单项widget, 例如, 一列图文列表, 每个列表的子项的widget都是由这个widget展示的, 对于这个widget, 你需要传入单个图文widget的引用, 从而能渲染这个widget的内容, 这个widget同样不需要存储状态, 它使用由父部件传入的数据, 来控制对用户的显示。

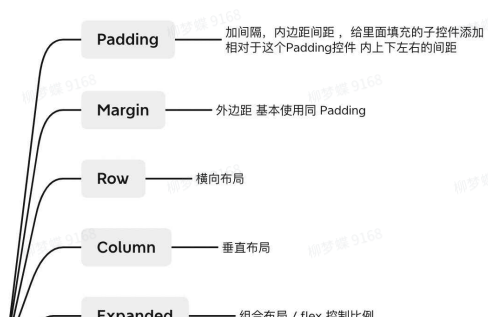
以上可以看出, **stateless是非动态的**, 它不依赖于除了传入数据以外的任何其他数据, 这意味着改变其显示的唯一方式, 就是改变传入其构造函数的参数。

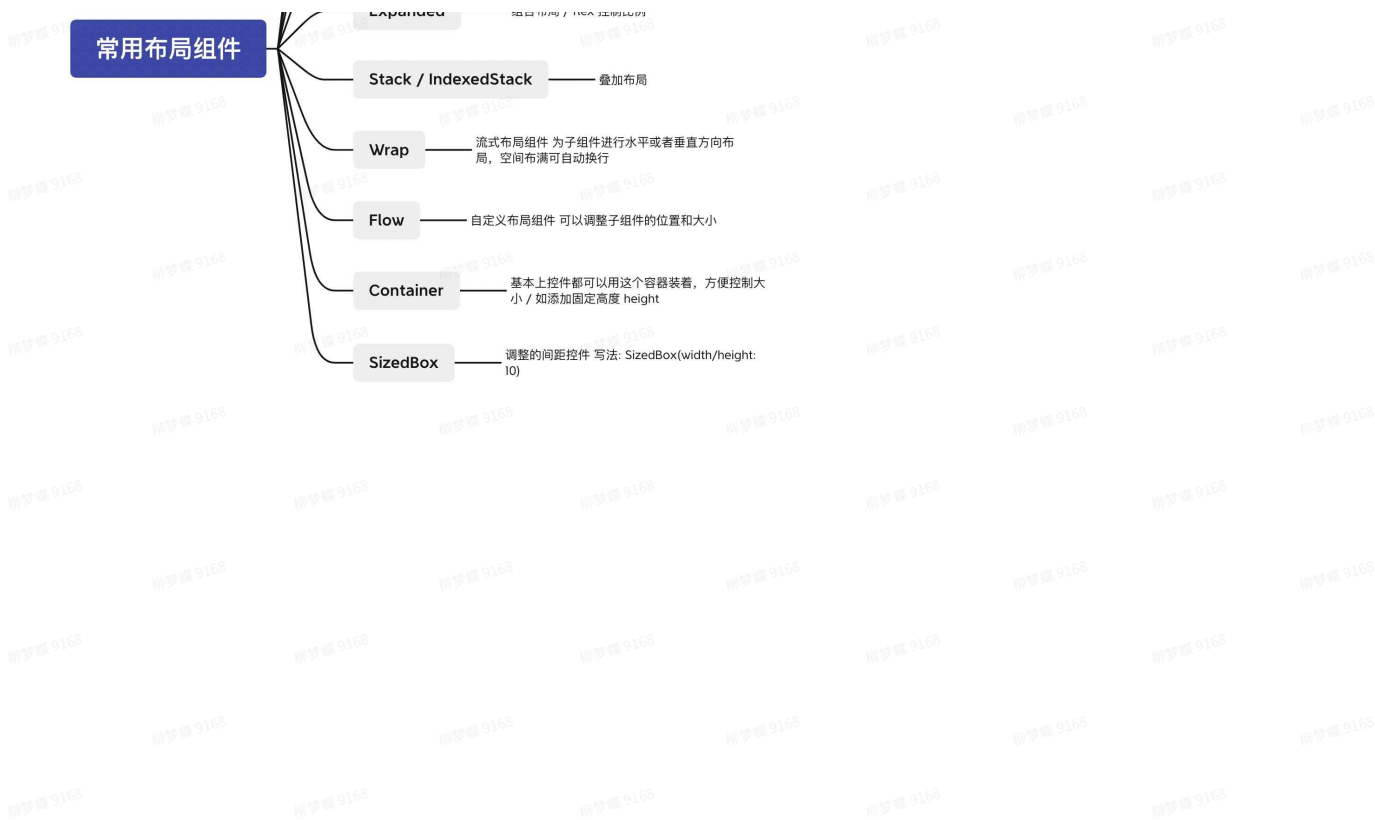
什么时候用stateful部件呢?

Stateful 是动态的, 他们允许我们创建一个能动随时间改变内容的widget, 并且不依赖于其初始化时被传入的静态状态, 可以随着用户的输入, 各种形式的异步回包或其他形式的状态变化而变化。



常用布局组件





编写第一个Flutter应用

1. Hello World

为了简化开发, Flutter 提供了脚手架, 即Scaffold。一般来说, 总是定义一个 Scaffold 当作实参传入到 MaterialApp 的 home 属性。换句话说, 一个 MaterialApp 总是绑定一个 Scaffold。Scaffold定义了一个 UI 框架, 这个框架包含 头部导航栏, body, 右下角浮动按钮, 底部导航栏等。

```
import 'package:flutter/material.dart';

class HelloWorldPage extends StatelessWidget {
  const HelloWorldPage({Key? key}) : super(key: key);
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Colors.black,
      ), // AppBar
      resizeToAvoidBottomInset: false,
      backgroundColor: Colors.white,
      body: const Center(
        child: Text('Hello World!'),
      ), // Center
    ); // Scaffold
  }
}
```

2. 路由

静态路由

在Flutter中有着两种路由跳转的方式，一种是静态路由，在创建时就已经明确知道了要跳转的页面和值。另一种是动态路由，跳转传入的目标地址和要传入的值都可以是动态的。

```
void main(){ runApp(new MaterialApp()); }
```

在runApp方法中需要传入一个MaterialApp的Widget，但是我们基本用到的都是home属性，但是其实MaterialApp方法里面有着很多的参数，其中routes参数就是定义路由的参数。

routes需要传入类型的Map，第一个参数是目标路由的名称，第二个参数就是你要跳转的页面。

```
routes: <String, WidgetBuilder>{  
  '/Hello World':(context)=> const HelloWorldPage(),  
  '/Todo List':(context)=> const TodoPage(),  
  '/Text Field':(context)=> const TextFieldPage(),  
  '/Todo Complete':(context)=> const TodoCompletePage(todoList: [],),  
  '/Todo Un Complete':(context)=> const TodoUnCompletePage(todoList: [],),  
  '/Login':(context)=> const LoginPage(),  
  '/List':(context)=> const ListPage(),  
}
```

动态路由

在Navigator中还有一个方法是push () 方法，需要传入一个Route对象，在Flutter中我们可以使用PageRouteBuilder来构建这个Route对象。

路由跳转

Flutter 中页面跳转通过Navigator和Route来实现

1，常规路由页面跳转

- 跳转到 `JumpPage()`；，需要用 `push` 方法，将 `JumpPage()`；入栈

```
1 Navigator.of(context).push(  
2   MaterialPageRoute(builder: (context) {  
3     return JumpPage(); //返回的是需要跳转单页面  
4   },));
```

- 返回上一级，出栈

```
1 //返回  
2 Navigator.of(context).pop(this);
```

2，命名路由页面跳转

路由名称通常使用路径结构: `"/a/b/c"`, 主页默认为 `"/"`。

`Navigator.pushNamed(context, '/xxxx');`, 需要调用这个push方法通常写法如下:

```
1 class MyApp extends StatelessWidget {
2   @override
3   Widget build(BuildContext context) {
4     return MaterialApp(
5       title: 'Flutter Demo',
6       theme: ThemeData(primarySwatch: Colors.blue, accentColor: Colors.blue),
7       home: MainTab(),
8       routes: {
9         //定义 /jump 对应 JumpPage(); 页面
10        '/jump':(context)=>JumpPage(),
11      }
12    );
13  }
14 }
```

3. 创建一个无限滚动ListView

4. 输入框textField

自己动手做一个todoList

样式自定

实现待办的添加 待办的状态修改 已完成和没完成待办的查看 一键完成所有待办

涉及内容点: StatefulWidget 路由跳转 widget传参 textField使用

参考demo







以上是mac环境实战

附: [windows安装flutter环境 踩坑记录](#)